

Unidad 1: Pensamiento Abstracto y TDA

Clase 1 – Pensar antes de programar: ¿qué es un TDA?

Objetivo

Comprender el concepto de TDA y poder leer/escribir una especificación básica y su “traducción” a Java a nivel interfaz/contrato

Tema

- Concepto de TDA.
- Especificación vs implementación.
- Operaciones y precondiciones.
- Notación básica.
- Java como herramienta (no como objetivo).

Introducción: aprender a pensar antes de programar

La programación no comienza escribiendo código. Comienza comprendiendo un problema, identificando qué información interviene y cómo debe organizarse esa información para poder operar sobre ella de manera coherente y eficiente. Esta afirmación puede parecer evidente, pero en la práctica muchos errores de diseño surgen justamente por omitir este paso.

En Programación I se trabajó principalmente con estructuras de control (condicionales, ciclos) y tipos de datos básicos. En esta materia el eje cambia: el foco ya no está únicamente en las instrucciones, sino en la **organización de los datos**. El objetivo es aprender a pensar estructuras antes de implementarlas.

Ese modo de razonamiento se denomina pensamiento abstracto. Abstraer implica quedarse con lo esencial de un problema y posponer detalles secundarios hasta que sean necesarios. Cuando abstraemos, no negamos los detalles; simplemente los separamos del modelo conceptual inicial.

El concepto central que formaliza esta forma de pensar en informática es el de **Tipo de Dato Abstracto (TDA)**.

¿Qué es un tipo de dato? Volver a lo básico

Antes de comprender qué es un TDA, debemos revisar qué significa “tipo de dato”.

En los lenguajes de programación existen tipos básicos como int, boolean, double o char. Un tipo de dato define dos aspectos fundamentales: el conjunto de valores que puede tomar y las operaciones que pueden realizarse sobre esos valores.

Por ejemplo, el tipo boolean puede tomar únicamente dos valores: true y false. No puede tomar 3, ni “hola”, ni -1. Esos valores simplemente no pertenecen a su dominio. Sobre los valores booleanos pueden aplicarse operaciones lógicas como AND (&&), OR (||) y NOT (!).

El tipo int, por su parte, representa números enteros. En términos matemáticos ideales podríamos pensar en todos los números enteros (... , -2, -1, 0, 1, 2, ...), aunque en la práctica el lenguaje establece un rango finito debido a la representación en memoria. Sobre esos valores pueden realizarse operaciones aritméticas como suma, resta y comparación.

Por lo tanto, un tipo de dato no es simplemente “una variable”. Es una definición formal de qué valores existen y qué operaciones tienen sentido sobre ellos.

Hasta aquí, los tipos vienen dados por el lenguaje. Sin embargo, los problemas reales rara vez se resuelven únicamente con números sueltos o valores booleanos.

Cuando los tipos básicos no son suficientes

Consideremos un sistema de turnos en un hospital. Las personas llegan, reciben un número y son atendidas en el orden de llegada. ¿Cómo modelaríamos este comportamiento?

Podríamos utilizar un arreglo y manejar manualmente índices. Podríamos usar múltiples variables y escribir lógica adicional para mantener el orden. Sin embargo, antes de pensar en código conviene formular una pregunta más profunda: ¿qué estructura conceptual describe este comportamiento?

La respuesta es que estamos frente a un modelo donde el primero que entra es el primero que sale. Ese comportamiento tiene un nombre en informática: **cola**.

Notemos algo importante. Antes de hablar de arreglos, memoria o estructuras internas, ya hemos identificado el modelo conceptual. Ese modelo describe cómo se comportan los datos. Y ese es el punto de partida.

Lo mismo ocurre con el historial de acciones de un editor de texto. Cada acción se guarda y cuando el usuario presiona “Deshacer”, se revierte la última acción realizada. En este caso, el último en entrar es el primero en salir. Este comportamiento corresponde al modelo de **pila**.

En ambos casos, el problema no se entiende mejor escribiendo código inmediatamente, sino identificando la estructura que modela el comportamiento.

Definición de Tipo de Dato Abstracto

Un Tipo de Dato Abstracto es una descripción formal del comportamiento de un conjunto de datos y de las operaciones que pueden realizarse sobre ellos, sin especificar cómo se implementan internamente.

El término “abstracto” indica precisamente que se omiten los detalles de implementación. El TDA se concentra en el “qué” y no en el “cómo”.

Por ejemplo, una cola se define por las operaciones que permite: inicializarla, agregar un elemento, quitar el primero, consultar el primero y verificar si está vacía. Esa descripción no menciona arreglos, nodos ni memoria dinámica. Sin embargo, define completamente el comportamiento observable.

Este enfoque permite separar dos niveles de razonamiento: el diseño conceptual y la implementación concreta.

Especificación e implementación: una separación esencial

La especificación de un TDA describe su comportamiento. Indica qué operaciones existen, qué parámetros reciben, qué valor devuelven y bajo qué condiciones pueden ejecutarse.

Por ejemplo, en una pila las operaciones típicas son: inicializar, apilar, desapilar, consultar el elemento superior y verificar si está vacía. La operación “desapilar” solo puede ejecutarse si la pila no está vacía. Esta condición previa se denomina **precondición**.

La implementación, en cambio, describe cómo se logra ese comportamiento. Una pila puede implementarse mediante un arreglo, una lista enlazada u otra estructura equivalente. Todas esas implementaciones deben respetar la misma especificación.

Esta separación es uno de los principios fundamentales de la Ingeniería de Software moderna. Permite modificar la implementación sin alterar el resto del sistema, siempre que se mantenga el contrato definido por la especificación.

Precondiciones y contrato

Una precondición es una condición que debe cumplirse antes de ejecutar una operación. Por ejemplo, no tiene sentido quitar un elemento de una pila vacía. Por lo tanto, la operación “desapilar” requiere como precondición que la pila no esté vacía.

Este concepto introduce la idea de contrato. El usuario del TDA se compromete a respetar las precondiciones. La implementación se compromete a garantizar el comportamiento especificado si esas condiciones se cumplen.

Pensar en términos de contrato es pensar en términos profesionales. Significa que cada módulo del sistema tiene responsabilidades claras.

Expresión en Java: la interfaz como contrato

En Java, la especificación de un TDA puede expresarse mediante una interfaz. Una interfaz es una construcción del lenguaje que declara métodos sin definir su implementación. Es, en términos conceptuales, un contrato formal.

Por ejemplo, la especificación de una pila podría expresarse así:

```
public interface PilaTDA {
    void InicializarPila();
    void Apilar(int x);
    void Desapilar();    // Requiere que la pila no esté vacía
    int Tope();          // Requiere que la pila no esté vacía
    boolean PilaVacía();
}
```

Aquí no se define cómo se almacenan los datos ni qué estructura se utiliza. Solo se establecen las operaciones disponibles.

En el desarrollo práctico utilizaremos un entorno de desarrollo integrado (IDE, por sus siglas en inglés, Integrated Development Environment). Un IDE es un programa que facilita la escritura, compilación y ejecución del código. Eclipse es uno de los IDE que emplearemos en la materia. Sin embargo, es importante destacar que el IDE es solo una herramienta; el objetivo no es aprender a usar Eclipse, sino comprender los conceptos.

Contexto histórico: por qué este concepto fue necesario

En la década de 1960 el software comenzó a crecer en complejidad. Los sistemas militares, bancarios e industriales dependían cada vez más de programas extensos y difíciles de mantener. El código y los datos estaban fuertemente acoplados, lo que hacía que cualquier cambio generara efectos en cadena.

Esta situación llevó a lo que se conoció como la Crisis del Software. En 1968, una conferencia organizada por la OTAN puso de manifiesto la necesidad de tratar el desarrollo de software como una disciplina de ingeniería.

En ese contexto surgieron principios como el ocultamiento de información y, poco después, la formalización del concepto de Tipo de Dato Abstracto. La idea de separar especificación e implementación permitió construir sistemas más modulares, mantenibles y confiables.

Cierre conceptual

Toda la materia puede resumirse en una afirmación clásica:

Un programa está compuesto por estructuras de datos y algoritmos.

Las estructuras de datos organizan la información.

Los algoritmos la procesan.

Elegir correctamente la estructura adecuada simplifica el diseño del algoritmo. Pensar en términos de Tipos de Datos Abstractos es el primer paso para desarrollar software con criterio profesional.

En esta primera clase hemos dado el paso fundamental: comprender que antes de programar es necesario pensar la estructura de los datos. A partir de aquí, comenzaremos a profundizar en la especificación formal y luego en la implementación de cada TDA.

Bibliografía

Liskov, B., & Zilles, S. (1974). *Programming with Abstract Data Types*. ACM SIGPLAN Notices, 9(4), 50–59.

— Artículo fundacional donde se formaliza el concepto de Abstract Data Type.

Parnas, D. L. (1972). *On the Criteria To Be Used in Decomposing Systems into Modules*. Communications of the ACM, 15(12), 1053–1058.

— Introduce el principio de ocultamiento de información, base conceptual del TDA.

Wirth, N. (1976). *Algorithms + Data Structures = Programs*. Prentice Hall.

— Obra clásica que establece la relación fundamental entre estructuras y algoritmos.

Knuth, D. E. (1968–1973). *The Art of Computer Programming* (Vols. 1–3). Addison-Wesley.

— Desarrollo formal del análisis de algoritmos y estructuras fundamentales.

Dijkstra, E. W. (1976). *A Discipline of Programming*. Prentice Hall.

— Fundamentación rigurosa del razonamiento formal en programación.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.

— Consolidación del diseño modular y programación basada en abstracciones.

Martin, R. C. (2000). *Design Principles and Design Patterns*.

— Desarrollo de principios como dependencia hacia abstracciones (base moderna del uso de interfaces).

Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.

— Enfatiza diseño modular, separación de responsabilidades y contratos claros.

Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley.

— Uso profesional de interfaces y diseño basado en contratos en Java.

Oracle. (2023). *The Java Tutorials – Interfaces and Inheritance*.

— Documentación oficial sobre el uso de interfaces como contratos en Java.

Cuadrado Estrebou, M. F., & Trutner, G. H. *Programación II – Apuntes de cátedra*.

Facultad de Ingeniería y Ciencias Exactas, Universidad Argentina de la Empresa.

— Material base utilizado para el desarrollo conceptual de la unidad.